LAPORAN PRAKTIKUM
STRUKTUR DATA
STACK

Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadirat Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang Stack menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa lapporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep Stack.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 2

2.1 Dasar Teori 2

2.1.1 Stack 2

2.1.2 Jenis Struktur Data Stack 2

2.1.3 Operasi Dasar Pada Stack 2

2.2 Langkah Kerja (Kode Program) 2

2.2.1 Kelas latihanStack_2511533008 2

2.2.2 Kelas NilaiMaksimum_2511533008 3

2.2.3 Kelas stackArray_2511533008 5

2.2.4 Kelas stackArrayDriver_2511533008 6

2.2.5 Kelas SiswaStack_2511533008 7

2.2.6 Kelas StackPostfix_2511533008 9

2.3 Output Program 10

2.3.1 Output latihanStack_2511533008 10

2.3.2 Output NilaiMaksimum_2511533008 10

2.3.3 Output stackArrayDriver_2511533008 10

2.3.4 Output SiswaStack_2511533008 11
2.3.5 Output StackPostfix_2511533008..... 11
BAB III PENUTUP 12
3.1 Kesimpulan 12
DAFTAR PUSTAKA 13
TUGAS 14
1. Website_2511533008 14
2. Browser_2511533008 15
3. Output Tugas 16

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berfungsi untuk mengatur dan mengelola data agar dapat diproses secara efisien. Salah satu bentuk struktur data linear yang penting adalah Stack, yang bekerja dengan prinsip *Last In First Out (LIFO)*, yaitu elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan.

Dalam kehidupan sehari-hari, konsep Stack dapat ditemukan pada berbagai aplikasi, seperti sistem undo/redo pada perangkat lunak, riwayat browser (browser history), serta proses pemanggilan fungsi secara rekursif. Oleh karena itu, pemahaman mengenai cara kerja Stack sangat penting bagi mahasiswa untuk menghubungkan teori dengan praktik nyata.

1.2 Tujuan Praktikum

1. Mengetahui konsep dasar stack.
2. Mengimplementasikan operasi dasar stack seperti push, peek, pop, isEmpty dalam bahasa pemrograman.

1.3 Manfaat Praktikum

1. Memahami konsep dasar stack.
2. Dapat mengimplementasikan operasi dasar stack dalam bahasa pemrograman.

BAB II
PEMBAHASAN

2.1 Dasar Teori

2.1.1 Stack

Stack (tumpukan) adalah struktur data linier yang mengikuti urutan tertentu di mana operasi dilakukan. Stack bersifat LIFO (Last In First Out) yaitu elemen terakhir yang dimasukkan adalah yang pertama dikeluarkan.

2.1.2 Jenis Struktur Data Stack

1. Stack Ukuran Tetap : memiliki ukuran yang tetap dan tidak dapat bertambah atau menyusut secara dinamis. Jika stack penuh dan ada upaya untuk menambahkan elemen kedalamnya, maka akan terjadi kesalahan overflow. Jika stack kosong dan ada upaya untuk menghapus elemen darinya, maka akan terjadi kesalahan overflow.
2. Stack Ukuran Dinamis : Dapat bertambah atau berkurang secara dinamis. Ketika stack penuh, secara otomatis meningkatkan ukurannya untuk mengakomodasi elemen baru, dan ketika stack kosong, stack mengurangi ukurannya. Jenis stack ini diimplementasikan menggunakan linked list, karena memungkinkan untuk mengubah ukuran stack dengan mudah.

2.1.3 Operasi Dasar Pada Stack

1. Push (menambahkan elemen baru pada tumpukan)
2. Pop (menghapus dan mengembalikan elemen teratas dari tumpukan)
3. Peek (mengembalikan elemen teratas pada stack)
4. isEmpty (memeriksa apakah tumpukan kosong)
5. size (menemukan jumlah elemen dalam tumpukan)

2.2 Langkah Kerja (Kode Program)

2.2.1 Kelas latihanStack_2511533008

```
package pekan3_2511533008;
import java.util.Stack;
```

```java
public class latihanStack_2511533008 {
	public static void main (String[] args) {
		Stack<Integer> s = new Stack<Integer>();
		s.push(42);
		s.push(-3);
		s.push(17);
		System.out.println("nilai stack = " + s);
		System.out.println("nilai pop = " + s.pop());
		System.out.println("nilai stack setelah pop = " + s);
		System.out.println("nilai peek = " + s.peek());
		System.out.println("nilai stack setelah peek = " + s);
	}

}
```

Penjelasan kode program:

1. Stack<Integer> s = new Stack<Integer>(); membuat stack.
2. s.push(42)

   s.push(-3)

   s.push(17)

   untuk menambah elemen pada stack.
3. System.out.println("Nilai Stack = " + s);

   Menampilkan nilai stack.
4. s.pop() menghapus dan mengembalikan nilai paling atas.
5. s.peek() mengambil nilai paling atas tanpa menghapusnya.

2.2.2 Kelas NilaiMaksimum_2511533008

```java
package pekan3_2511533008;
import java.util.Stack;

public class NilaiMaksimum_2511533008 {
	public static int max_2511533008(Stack<Integer> s) {
		Stack<Integer> backup_2511533008 = new
Stack<Integer>();
		int maxValue_2511533008 = s.pop();
		backup_2511533008.push(maxValue_2511533008);
		while(!s.isEmpty()) {
			int next_2511533008 = s.pop();
			backup_2511533008.push(next_2511533008);
			maxValue_2511533008 =
Math.max(maxValue_2511533008, next_2511533008);
		}
		while(!backup_2511533008.isEmpty()) {
```

```
                    s.push(backup_2511533008.pop());
            }
            return maxValue_2511533008;
      }

      public static void main(String[] args) {
            Stack<Integer> s = new Stack<Integer>();
            s.push(70);
            s.push(12);
            s.push(20);
            System.out.println("isi stack " + s);
            System.out.println("Stack Teratas " + s.peek());
            System.out.println("Nilai Maksimum " +
max_2511533008(s));

      }

}
```

Penjelasan kode program:

1. Stack<Integer> backup_2511533008 = new Stack<Integer>();

   Membuat stack.

2. int maxValue_2511533008 = s.pop();

   elemen awal diambil sebagai nilai maksimum awal.

3. Loop semua elemen

   while(!s.isEmpty())

   selama stack terdapat isi:

   ambil elemen, simpan ke backup dan bandingkan dengan nilai maksimum.

   int next_2511533008 = s.pop();
   maxValue_2511533008 = Math.max(maxValue_2511533008, next_2511533008);

4. while(!backup_2511533008.isEmpty()) {

   s.push(backup_2511533008.pop());

   }

   Mengembalikan isi stack seperti semula.

5. return maxValue_2511533008;

   megembalikan nilai maksimum.

2.2.3 Kelas stackArray_2511533008

```
package pekan3_2511533008;

public class stackArray_2511533008 {
	static final int MAX = 1000;
	int top;
	int a[] = new int[MAX];
	boolean isEmpty_2511533008()
	{
		return (top < 0);
	}
	stackArray_2511533008()
	{
		top = -1;
	}
	boolean push_2511533008(int x)
	{
		if (top >= (MAX - 1)) {
			System.out.println("Stack Overflow");
			return false;
		}
		else {
			a[++top] = x;
			System.out.println(x + " dimasukkan dalam
stack");
			return true;
		}
	}
	int pop_2511533008()
	{
		if (top < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int x = a[top--];
			return x;
		}
	}
	int peek_2511533008()
	{
		if (top < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int x = a[top];
			return x;
		}
	}
	void print_2511533008 () {
		for (int i = top; i > -1; i--) {
```

```
                    System.out.print(" " + a[i]);
            }
    }

}
```

Penjelasan kode program :

1. static final int MAX = 1000;
   int top;
   int a[] = new int[MAX];

   MAX → menunjukkan kapasitas maksimum stack

   top → penunjuk posisi tertas stack

   a[] → array sebagai tempat penyimpanan data
2. stackArray_2511533008()
   {
   top = -1;
   }

   Menandakan stack masi kosong
3. Method push_2511533008 () menambahkan elemen kedalam stack.
4. Method pop_2511533008() menghapus data yang paling atas.
5. Method peek_2511533008() menampilkan data teratas.

2.2.4 Kelas stackArrayDriver_2511533008

```
package pekan3_2511533008;

public class stackArrayDriver_2511533008 {

      public static void main(String[] args) {
            stackArray_2511533008 s = new stackArray_2511533008();
            s.push_2511533008(10);
            s.push_2511533008(20);
            s.push_2511533008(30);
            System.out.println(s.pop_2511533008() + " dikeluarkan
dari stack");
            System.out.println("Elemen teratas adalah : " +
s.peek_2511533008());
            System.out.print("Elemen pada stack :");
            s.print_2511533008();

      }
```

```
}
```

Penjelasan kode program:

Pada stackArrayDriver_2511533008 ini kita menggunakan method yang telah di buat pada kelas stackArray_2511533008.

2.2.5 Kelas SiswaStack_2511533008

```
package pekan3_2511533008;
import java.util.ArrayList;

class Siswa_2511533008 {
	String nama;
	int nim;

	public Siswa_2511533008(String nama, int nim) {
		this.nama = nama;
		this.nim = nim;
	}
	//override
	public String toString() {
		return "Nim: " + nim + ", Nama: " + nama;
	}
}
public class SiswaStack_2511533008 {
	private ArrayList<Siswa_2511533008> stack;

	public SiswaStack_2511533008() {
		stack = new ArrayList<>();
	}

	public void push (Siswa_2511533008 mhs) {
		stack.add(mhs);
	}

	public Siswa_2511533008 pop() {
		if (!isEmpty()) {
			return stack.remove(stack.size() - 1);
		}
		return null;
	}

	public Siswa_2511533008 peek() {
		if (!isEmpty()) {
			return stack.get(stack.size() - 1);
		}
		return null;
	}
```

```
	public boolean isEmpty() {
		return stack.isEmpty();
	}

	public void tampilkanSiswa_2511533008() {
		for (int i = stack.size()-1; i >= 0; i--) {
			System.out.println(stack.get(i));
		}
	}

	public static void main (String[] args) {
		SiswaStack_2511533008 studentStack_2511533008 = new
SiswaStack_2511533008();

		Siswa_2511533008 mhs1 = new Siswa_2511533008("Ali", 1);
		Siswa_2511533008 mhs2 = new Siswa_2511533008("Boby",
2);
		Siswa_2511533008 mhs3 = new Siswa_2511533008("Charles",
3);

		studentStack_2511533008.push(mhs1);
		studentStack_2511533008.push(mhs2);
		studentStack_2511533008.push(mhs3);

		System.out.println("Siswa di dalam stack: ");
		studentStack_2511533008.tampilkanSiswa_2511533008();

		System.out.println("Siswa teratas " +
studentStack_2511533008.peek());
		System.out.println("mengeluarkan siswa teratas dari
stack: " + studentStack_2511533008.pop());
		System.out.println("daftar siswa satelah di pop :");
		studentStack_2511533008.tampilkanSiswa_2511533008();
	}
}
```

Penjelasan kode program:

1. class Siswa_2511533008 {

   String nama;

   int nim;

   class ini merepresentasikan 2 objek yaitu nama dan nim.

2. Terdapat konstruktor sebagai inisialisasi objek yang dibuat

   public Siswa_2511533008(String nama, int nim)

3. Terdapat override toString yang membuat tampilan lebih rapi saat dicetak

   return "Nim: " + nim + ", Nama: " + nama;

4. private ArrayList<Siswa_2511533008> stack;

   menggunakan ArrayList untuk penyimpanan data stack.

5. Terdapat operasi push(), pop(), peek(), dan isEmpty() yang merupakan operasi pada stack.

2.2.6 Kelas StackPostfix_2511533008

```
package pekan3_2511533008;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511533008 {
	public static int postfixEvaluate_2511533008(String
expression) {
		Stack<Integer> s = new Stack<Integer>();
		Scanner input_2511533008 = new Scanner(expression);
		while (input_2511533008.hasNext()) {
			if (input_2511533008.hasNextInt()) {	// an
operand (integer)
				s.push(input_2511533008.nextInt());
			} else {				// an operator
				String operator = input_2511533008.next();
				int operand2 = s.pop();
				int operand1 = s.pop();
				if (operator.equals("+")) {
					s.push(operand1 + operand2);
				} else if (operator.equals("-")) {
					s.push(operand1 - operand2);
				} else if (operator.equals("*")) {
					s.push(operand1 * operand2);
				} else {
					s.push(operand1 / operand2);
				}
			}
		}
		input_2511533008.close();
		return s.pop();
	}

	public static void main(String[] args) {
		System.out.println("hasil postfix = " +
postfixEvaluate_2511533008("5 2 4 * + 7 -"));


	}
```

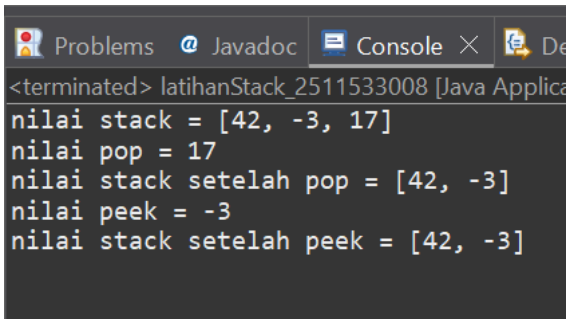
```
}
```

Penjelasan kode program:

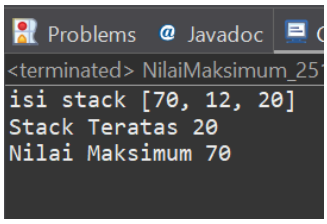
Program ini mengimplementasikan evaluasi ekspresi postfix menggunakan struktur data stack. Setiap operand dimasukkan ke dalam stack, sedangkan operator akan mengambil dua operand teratas untuk dilakukan operasi aritmatika sesuai operator tersebut. Hasil dari operasi kemudian dimasukkan kembali ke dalam stack hingga diperoleh hasil akhir.

2.3 Output Program

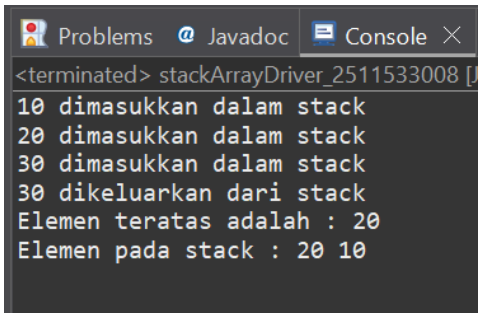
2.3.1 Output latihanStack_2511533008



2.3.2 Output NilaiMaksimum_2511533008



2.3.3 Output stackArrayDriver_2511533008



2.3.4 Output SiswaStack_2511533008

```
Problems  @ Javadoc  Console ×  Declaration  Git Staging
<terminated> SiswaStack_2511533008 [Java Application] C:\Users\asusvivobook\.p2\pool\
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa satelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

2.3.5 Output StackPostfix_2511533008

```
Problems  @ Javadoc  Console ×
<terminated> StackPostfix_2511533008 [Java A
hasil postfix = 6
```

BAB III
PENUTUP

3.1 Kesimpulan

Melalui praktikum struktur data Stack, mahasiswa dapat memahami bahwa Stack merupakan salah satu struktur data linear yang bekerja dengan prinsip *Last In First Out (LIFO)*, di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Implementasi operasi dasar seperti *push*, *pop*, *peek*, dan *isEmpty*. Praktikum ini juga menunjukkan relevansi konsep Stack dalam kehidupan nyata, misalnya pada sistem riwayat browser, fitur *undo/redo*, maupun pemanggilan fungsi rekursif, serta algoritma postfix. Dengan demikian, praktikum ini tidak hanya memperkuat pemahaman teori, tetapi juga menumbuhkan kemampuan analitis mahasiswa dalam memilih struktur data yang tepat untuk mendukung efisiensi algoritma dan pengembangan perangkat lunak.

DAFTAR PUSTAKA

[1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.

[2] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.

[3] Departemen Informatika Universitas Andalas, *Pekan 4 Stack*, Materi Kuliah Struktur Data, Universitas Andalas, 2026.

TUGAS

1. Website_2511533008

```java
package pekan3_2511533008;
import java.util.Stack;

public class Website_2511533008 {
    private String judul_3008;
    private String url_3008;

    public Website_2511533008 (String judul_3008, String url_3008) {
        this.judul_3008 = judul_3008;
        this.url_3008 = url_3008;
    }
    public String getJudul_3008() {
        return judul_3008;
    }
    public String getUrl_3008() {
        return url_3008;
    }
    public void setJudul_3008(String judul_3008) {this.judul_3008 = judul_3008;}
    public void setUrl_3008(String url_3008) {this.url_3008 = url_3008;}

    // override
    public String toString() {
        return judul_3008 + " (" + url_3008 + ")";
    }


}

```

Pada kelas ini terdapat :

1. Konstruktor yang menginisialisasi judul_3008 dan url_3008 sebagai string.
2. Terdapat getter yang berfunsi mengambil nilai dari atribut private.
3. Terdapat setter untuk mengubah data.
4. Method toString() untuk tampilan cetak data.

2. Browser_2511533008

```java
package pekan3_2511533008;
import java.util.Stack;
import java.util.Scanner;

public class Browser_2511533008 {

    public static void main(String[] args) {
        Stack<Website_2511533008> history_3008 = new Stack<>();
        Scanner sc = new Scanner (System.in);
        int pilihan_3008;
         do {
                System.out.println("\n=== Browser History NIM: 2511533008 ===");
                System.out.println("1. Kunjungi Website (Push)");
                System.out.println("2. Tombol Back (Pop)");
                System.out.println("3. Lihat Halaman Aktif (Peek)");
                System.out.println("4. Keluar");
                System.out.print("Pilihan: ");
                pilihan_3008 = sc.nextInt();
                sc.nextLine();

                switch (pilihan_3008) {
                case 1:
                    System.out.print("Masukkan Judul: ");
                    String judul_3008 = sc.nextLine();
                    System.out.print("Masukkan URL: ");
                    String url_3008 = sc.nextLine();
                    Website_2511533008 web_3008 = new Website_2511533008(judul_3008, url_3008);
                    history_3008.push(web_3008);
                    System.out.println("Berhasil mengunjungi halaman!");
                    break;

                case 2:
```

```java
                case 2:
                    if (!history_3008.isEmpty()) {
                        Website_2511533008 keluar_3008 = history_3008.pop();
                        System.out.println("Kembali dari halaman: " + keluar_3008);
                    } else {
                        System.out.println("Riwayat kosong, tidak bisa kembali!");
                    }
                    break;

                case 3:
                    if (!history_3008.isEmpty()) {
                        System.out.println("Halaman aktif: " + history_3008.peek());
                    } else {
                        System.out.println("Tidak ada halaman aktif!");
                    }
                    break;

                case 4:
                    System.out.println("Keluar dari browser...");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan_3008 != 4);

        sc.close();

    }

}
```

Penjelasan :

Pada program ini kita mengimplementasikan adt yang telah di buat pada kelas Website_2511533008.

3. Output Tugas

```
Problems  Javadoc  Console  Declaration  Git Staging
<terminated> Playlist_2511533008 [Java Application] C:\Users\asusvivobook\.p2\pool
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan : 1
Judul Lagu : Nas Teshbehlena
Penyanyi : Maher Zain
Durasi : 256
Musik berhasil ditambahkan
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan : 2

Playlist:
1. Nas Teshbehlena - Maher Zain (256 detik)
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan : 4
Jumlah total lagu : 1
```